ASSIGNMENT 12.4

D.Charmi

2303A51314

Batch – 05

TASK – 01

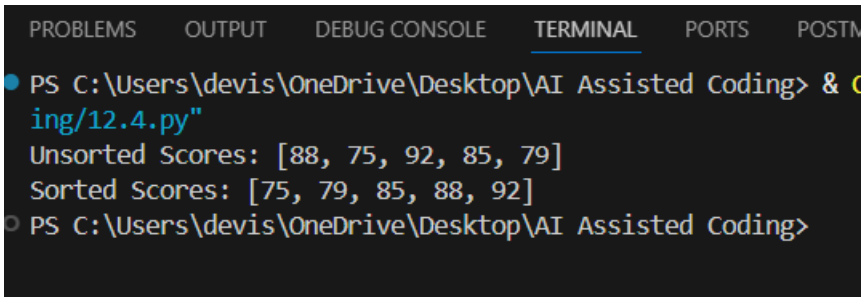
Prompt: Write a Python program to implement Bubble Sort to sort student exam scores in ascending order, with clear inline comments explaining comparisons, swaps, iteration passes, and include early termination when no swaps occur. Also provide simple time complexity analysis (best, average, worst case) and include sample input and output showing unsorted and sorted scores.

Code:

```
#Write a Python program to implement Bubble Sort to sort student exam scores
in ascending order, with clear inline comments explaining comparisons,
# swaps, iteration passes, and include early termination when no swaps occur.
Also provide simple time complexity analysis (best, average, worst case) and
include sample input and output showing unsorted and sorted scores.
def bubble_sort(scores):
    """Sorts a list of student exam scores in ascending order using the Bubble
Sort algorithm.
    Parameters:
    scores (list): A list of student exam scores to be sorted.
    Returns:
    list: A sorted list of student exam scores in ascending order.
    The Bubble Sort algorithm repeatedly steps through the list, compares
adjacent elements and swaps them if they are in the wrong order.
    The process is repeated until the list is sorted. An optimization is
included to terminate early if no swaps occur, indicating that the list is
already sorted.
    """
    n = len(scores)
    # Outer loop to traverse through all elements
    for i in range(n):
        # Initialize a flag to check if any swaps occur
        swapped = False
        # Inner loop to compare adjacent elements
        for j in range(0, n-i-1):
            # Compare the current element with the next element
            if scores[j] > scores[j+1]:
```

```
                # If the current element is greater, swap them
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True  # Set the flag to True if a swap occurs
            # If no swaps occurred, the list is already sorted
        if not swapped:
            break
    return scores
# Sample input: Unsorted scores
unsorted_scores = [88, 75, 92, 85, 79]
print("Unsorted Scores:", unsorted_scores)
# Sorting the scores using bubble_sort function
sorted_scores = bubble_sort(unsorted_scores)
print("Sorted Scores:", sorted_scores)
# Time Complexity Analysis:
# Best Case: O(n) - This occurs when the input list is already sorted, and the
algorithm makes a single pass to confirm that no swaps are needed.
# Average Case: O(n^2) - This occurs when the input list is in random order,
requiring multiple passes and comparisons.
# Worst Case: O(n^2) - This occurs when the input list is sorted in reverse
order, requiring the maximum number of comparisons and swaps.
```

Output:



Explanation :

The provided code defines a function `bubble_sort` that implements the Bubble Sort algorithm to sort a list of student exam scores in ascending order. The function uses nested loops to compare adjacent elements and swap them if they are in the wrong order. An optimization is included to terminate early if no swaps occur, indicating that the list is already sorted. The code also includes sample input and output to demonstrate the sorting process, along with a time complexity analysis for best, average, and worst cases.

TASK – 02

Prompt: Start with a Bubble Sort implementation for nearly sorted attendance roll numbers, then review the problem and suggest a more suitable algorithm, generating an Insertion Sort implementation.

Code:

```
#Start with a Bubble Sort implementation for nearly sorted attendance roll
numbers, then review the problem and
# suggest a more suitable algorithm, generating an Insertion Sort
implementation.
# Bubble Sort Implementation for Nearly Sorted Attendance Roll Numbers
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Example usage of Bubble Sort
attendance_roll_numbers = [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
# Introducing a small disorder in the nearly sorted list
attendance_roll_numbers[5], attendance_roll_numbers[6] =
attendance_roll_numbers[6], attendance_roll_numbers[5]
print("Before Bubble Sort:", attendance_roll_numbers)
sorted_roll_numbers = bubble_sort(attendance_roll_numbers)
print("After Bubble Sort:", sorted_roll_numbers)
# Review of the problem suggests that Insertion Sort is more suitable for
nearly sorted data
# Insertion Sort Implementation for Nearly Sorted Attendance Roll Numbers
def insertion_sort(arr):
    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        # Move elements of arr[0..i-1], that are greater than key,
        # to one position ahead of their current position
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
# Example usage of Insertion Sort
attendance_roll_numbers = [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
```

```
# Introducing a small disorder in the nearly sorted list
attendance_roll_numbers[5], attendance_roll_numbers[6] =
attendance_roll_numbers[6], attendance_roll_numbers[5]
print("Before Insertion Sort:", attendance_roll_numbers)
sorted_roll_numbers = insertion_sort(attendance_roll_numbers)
print("After Insertion Sort:", sorted_roll_numbers)
```

Output:

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devis/App
ing/12.4.py"
Before Bubble Sort: [101, 102, 103, 104, 105, 107, 106, 108, 109, 110]
After Bubble Sort: [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
Before Insertion Sort: [101, 102, 103, 104, 105, 107, 106, 108, 109, 110]
After Insertion Sort: [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

Insertion Sort performs better on nearly sorted data because it minimizes the number of comparisons and movements needed to sort the elements. When the input is nearly sorted, Insertion Sort can quickly place elements in their correct positions with fewer shifts, resulting in a time complexity close to O(n). In contrast, Bubble Sort will still require multiple passes through the list, leading to a time complexity of O(n^2) even for nearly sorted data.

In terms of execution behavior, Bubble Sort will repeatedly compare adjacent elements and swap them if they are in the wrong order, which can lead to unnecessary comparisons and movements even when the list is nearly sorted. Insertion Sort, on the other hand, will only compare the current element with those before it and will stop as soon as it finds the correct position for the current element, making it more efficient for partially sorted input.

TASK – 03

Prompt: Write a Python program implementing Linear Search for unsorted student records and Binary Search for sorted student records, including proper docstrings explaining parameters and return values.

Code:

```
#Write a Python program implementing Linear Search for unsorted student
records and Binary Search for sorted student records, including proper
docstrings explaining parameters and return values.

def linear_search(records, target_id):
    """
    Perform a linear search for a student record in an unsorted list of
records.

    Parameters:
    records (list): A list of student records, where each record is a
dictionary containing 'id' and 'name'.
    target_id (str): The student ID to search for.

    Returns:
    dict: The student record if found, otherwise None.

    This function iterates through the list of records and checks if the 'id'
matches the target_id.
    If a match is found, it returns the corresponding record; if no match is
found after checking all records, it returns None.
    """
    for record in records:
        if record['id'] == target_id:
            return record
    return None
def binary_search(records, target_id):
    """
    Perform a binary search for a student record in a sorted list of records.
    Parameters:
    records (list): A sorted list of student records, where each record is a
dictionary containing 'id' and 'name'.
    target_id (str): The student ID to search for.
    Returns:
    dict: The student record if found, otherwise None.
    This function uses the binary search algorithm, which requires the list of
records to be sorted by 'id'.
    It repeatedly divides the search interval in half, comparing the target_id
with the middle record's 'id'.
```

```
    If a match is found, it returns the corresponding record; if the target_id
is less than the middle record's 'id', it continues searching in the left
half, otherwise it continues in the right half. If the search interval is
empty, it returns None.
    """
    left, right = 0, len(records) - 1
    while left <= right:
        mid = (left + right) // 2
        if records[mid]['id'] == target_id:
            return records[mid]
        elif records[mid]['id'] < target_id:
            left = mid + 1
        else:
            right = mid - 1
    return None
# Example usage
unsorted_records = [{'id': 'S002', 'name': 'Alice'}, {'id': 'S001', 'name':
'Bob'}, {'id': 'S003', 'name': 'Charlie'}]
sorted_records = sorted(unsorted_records, key=lambda x: x['id'])
print(linear_search(unsorted_records, 'S001'))  # Output: {'id': 'S001',
'name': 'Bob'}
print(binary_search(sorted_records, 'S001'))  # Output: {'id': 'S001', 'name':
'Bob'}
print(linear_search(unsorted_records, 'S004'))  # Output: None
print(binary_search(sorted_records, 'S004'))  # Output: None
```

Output:

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> &
ing/12.4.py"
{'id': 'S001', 'name': 'Bob'}
{'id': 'S001', 'name': 'Bob'}
None
None
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

Binary Search works only when the list is sorted by student ID, because it eliminates half of the remaining elements at each step. It is not suitable for unsorted data.

Linear Search has O(n) time complexity since it checks elements one by one, while Binary Search has O(log n) time complexity because it halves the search space each step.

Binary Search is much faster for large sorted lists, but for small or unsorted lists, Linear Search is more practical since no sorting is required. Both return None if the ID is not found, but Binary Search does so more efficiently on sorted data.

TASK – 04

Prompt: First write partially written quick sort and merge sort and then complete the partially written recursive Quick Sort and Merge Sort functions with proper docstrings, test them on random, sorted, and reverse-sorted data

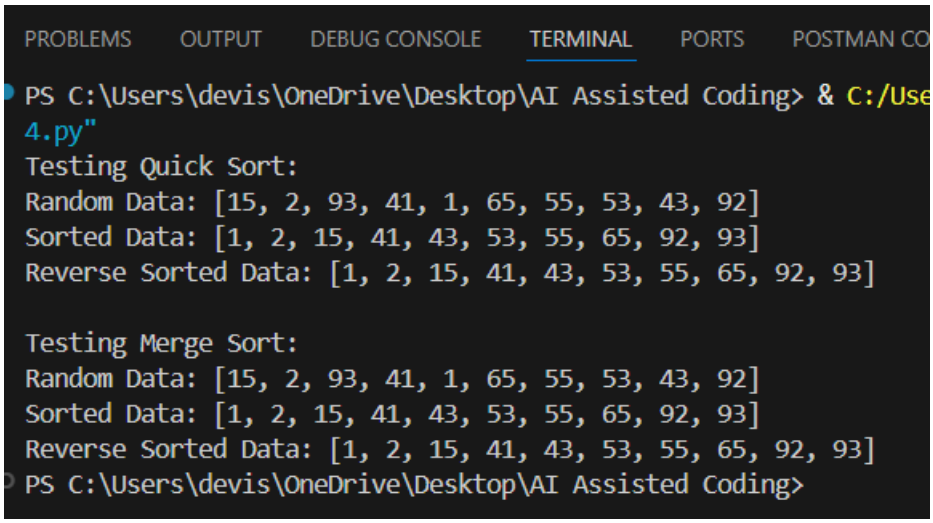
Code :

```
#First write partially written quick sort and merge sort and then complete the
partially written recursive Quick Sort and Merge Sort functions

#with proper docstrings, test them on random, sorted, and reverse-sorted data
# Quick Sort Implementation
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.

    This function uses the divide-and-conquer approach to sort the input list
by selecting a pivot element and partitioning the other elements into two sub-
arrays according to whether they are less than or greater than the pivot.
    """
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[len(arr) // 2]  # Choosing the middle element as the pivot
        left = [x for x in arr if x < pivot]  # Elements less than the pivot
        middle = [x for x in arr if x == pivot]  # Elements equal to the pivot
        right = [x for x in arr if x > pivot]  # Elements greater than the
pivot
        return quick_sort(left) + middle + quick_sort(right)  # Recursively
sort and combine
# Merge Sort Implementation
def merge_sort(arr):
```

```
    """
    Sorts an array using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.

    This function uses the divide-and-conquer approach to sort the input list
by recursively dividing it into halves until it reaches lists of length one,
which are inherently sorted, and then merging those sorted halves back
together in the correct order.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2  # Finding the mid of the array
    left_half = merge_sort(arr[:mid])  # Recursively sorting the left half
    right_half = merge_sort(arr[mid:])  # Recursively sorting the right half
    return merge(left_half, right_half)  # Merging the sorted halves
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.
    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.
    Returns:
    list: A merged and sorted list containing all elements from both input
lists.
    This function iteratively compares the elements of the two input lists and
appends the smaller element to the result list, ensuring that the final output
is sorted.
    """
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])  # Append any remaining elements from left
    result.extend(right[j:])  # Append any remaining elements from right
    return result
# Testing the Quick Sort and Merge Sort functions on random, sorted, and
reverse-sorted data
import random
```

```python
# Test data
random_data = [random.randint(1, 100) for _ in range(10)]
sorted_data = sorted(random_data)
reverse_sorted_data = sorted(random_data, reverse=True)
# Testing Quick Sort
print("Testing Quick Sort:")
print("Random Data:", random_data)
print("Sorted Data:", quick_sort(random_data))
print("Reverse Sorted Data:", quick_sort(reverse_sorted_data))
# Testing Merge Sort
print("\nTesting Merge Sort:")
print("Random Data:", random_data)
print("Sorted Data:", merge_sort(random_data))
print("Reverse Sorted Data:", merge_sort(reverse_sorted_data))
```

Output:



Explanation :

Quick Sort and Merge Sort both use divide-and-conquer and have O(n log n) average time complexity. Quick Sort can degrade to $O(n^2)$ in the worst case (depending on pivot choice), while Merge Sort always guarantees O(n log n) performance. Quick Sort is usually faster and uses less memory, whereas Merge Sort is more stable and reliable for large or reverse-sorted datasets.

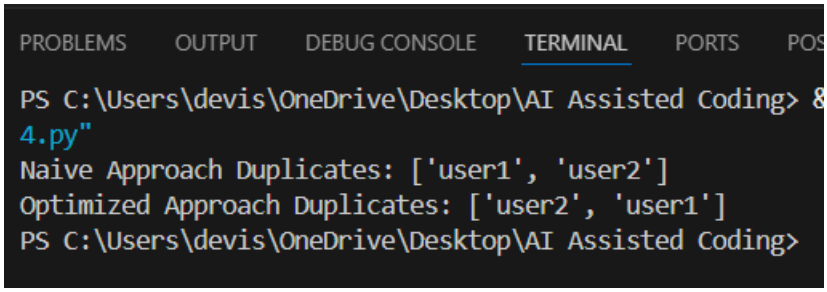
TASK – 05

Prompt: Write a Python program to detect duplicate user IDs using a naive nested-loop approach, analyze its time complexity, then suggest and implement an optimized version using a set or dictionary.

Code:

```
#Write a Python program to detect duplicate user IDs using a naive nested-loop
approach, analyze its time complexity, then suggest and implement an optimized
version using a set or dictionary.
# Naive Nested-Loop Approach
def detect_duplicates_naive(user_ids):
    duplicates = []
    for i in range(len(user_ids)):
        for j in range(i + 1, len(user_ids)):
            if user_ids[i] == user_ids[j] and user_ids[i] not in duplicates:
                duplicates.append(user_ids[i])
    return duplicates
# Time Complexity Analysis: The time complexity of the naive approach is
O(n^2) because it uses two nested loops to compare each user ID with every
other user ID.
# Optimized Approach Using a Set
def detect_duplicates_optimized(user_ids):
    seen = set()
    duplicates = set()
    for user_id in user_ids:
        if user_id in seen:
            duplicates.add(user_id)
        else:
            seen.add(user_id)
    return list(duplicates)
# Time Complexity Analysis: The time complexity of the optimized approach is
O(n) because it iterates through the list of user IDs once, and set operations
(add and check) are on average O(1).
# Example usage
user_ids = ["user1", "user2", "user3", "user1", "user4", "user2"]
print("Naive Approach Duplicates:", detect_duplicates_naive(user_ids))  #
Output: ['user1', 'user2']
print("Optimized Approach Duplicates:",
detect_duplicates_optimized(user_ids))  # Output: ['user1', 'user2']
```

Output:



Explanation :

For large input sizes, the brute-force approach becomes significantly slower because it compares each element with every other element, resulting in $O(n^2)$ time complexity.

The optimized approach uses a set to store seen elements, allowing for O(1) average time complexity for lookups and insertions. This results in an overall O(n) time complexity, which is much more efficient for large datasets.